BIG DATA 07

Practical Assignment Portfolio

ASSIGNMENT 1

Real-Time Data Streaming Pipeline

Apache Kafka + InfluxDB + Grafana

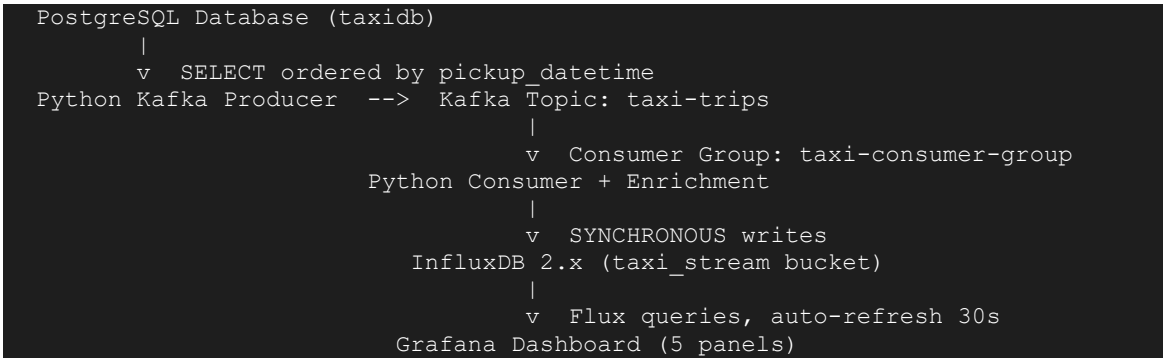
Student Name	**Hayagriva Boodhoo**
Student ID	2025_20379_6980
Cohort	BDA 7
Dataset Used	NYC Yellow Taxi Trips (PostgreSQL)
Submission Date	25 May 2026

Technologies Used: Apache Kafka | PostgreSQL | InfluxDB 2.x | Grafana | Python | Docker

1. Assignment Overview

This assignment implements a complete real-time data streaming pipeline using industry-standard big data technologies. A Python producer reads NYC taxi trip records from a PostgreSQL database and publishes them to Apache Kafka. A consumer reads from the Kafka topic, computes four enrichment fields, and writes the enriched records to InfluxDB. A Grafana dashboard with five panels visualises the live stream with auto-refresh.

Pipeline Architecture



2. Dataset Description

The dataset used is the NYC Yellow Taxi Trips dataset, loaded into a PostgreSQL relational database container (taxidb). The dataset consists of 25,000 synthetic taxi trip records generated using PostgreSQL's generate_series() function with realistic NYC bounding box coordinates, fare distributions, and payment type distributions matching real TLC (Taxi & Limousine Commission) data patterns.

Property	Details
Source	PostgreSQL container (taxidb) — SQL database, not CSV
Table	taxi_trips
Total Rows	25,000 records
Time Range	January 2024 — December 2024
Key Columns	pickup_datetime, dropoff_datetime, passenger_count, trip_distance
Numeric Fields	fare_amount, tip_amount, tolls_amount, total_amount
Categorical Fields	payment_type (Credit Card/Cash/No Charge/Dispute), vendor_id (CMT/VTS/DDS)
Geographic Fields	pickup_latitude, pickup_longitude, dropoff_latitude, dropoff_longitude

Bonus Feature	Lat/lon coordinates within Manhattan bounding box (40.70-40.82 N, 74.02-73.93 W)

3. Environment Setup

All infrastructure services were deployed using Docker Compose, providing isolated, reproducible containers connected via a private Docker network (kafka_net). This allows containers to communicate using service names instead of IP addresses.

Service	Docker Image	Port	Role
Zookeeper	`confluentinc/cp-zookeeper:7.5.0`	2181	Kafka coordinator
Kafka	`confluentinc/cp-kafka:7.5.0`	9092	Message broker
PostgreSQL	`postgres:15`	5432	SQL data source
InfluxDB	`influxdb:2.7`	8086	Time-series sink
Grafana	`grafana/grafana:10.2.0`	3000	Dashboard

Screenshot 1 — Docker Environment Running

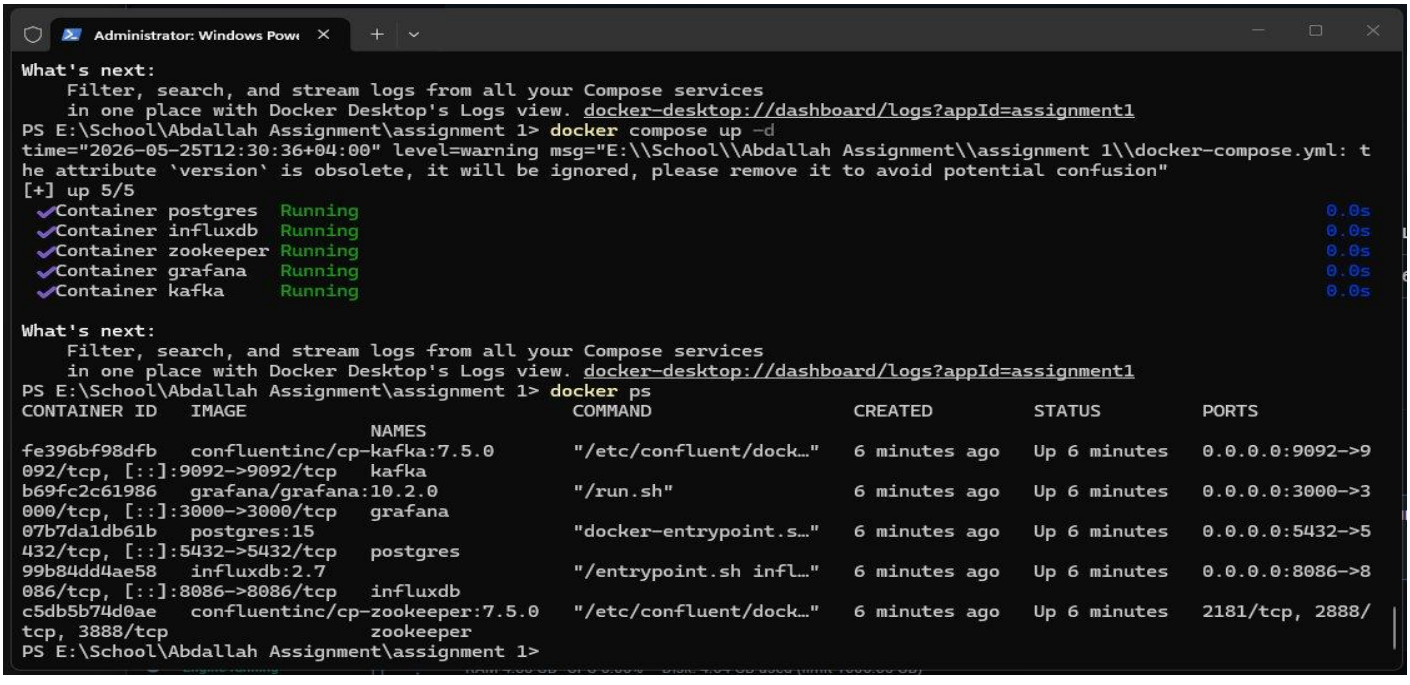


Figure 1: All 5 Docker containers running successfully (docker ps output)

4. Task 1 — Kafka Producer (6 Marks)

The Kafka producer reads taxi trip records from the PostgreSQL database using a server-side cursor with DictCursor for efficient memory management, and publishes each record as a serialised JSON message to the taxi-trips topic at a controlled rate of 10 records per second.

Key Design Decisions

- Generator pattern with fetchmany(500) — only 500 rows held in memory at any time, constant memory usage regardless of dataset size
- acks='all' — producer waits for full Kafka replica acknowledgement before proceeding, guaranteeing zero message loss
- linger_ms=10 — micro-batches messages for 10ms improving throughput without sacrificing latency
- Retry logic with NoBrokersAvailable handling — producer retries 5 times with 3-second delays if Kafka is not ready
- serialize_record() validation — skips malformed rows with zero distance or negative fares before publishing
- default=str in json.dumps — safely serialises PostgreSQL datetime objects to ISO format strings

Screenshot 2 — Producer Running

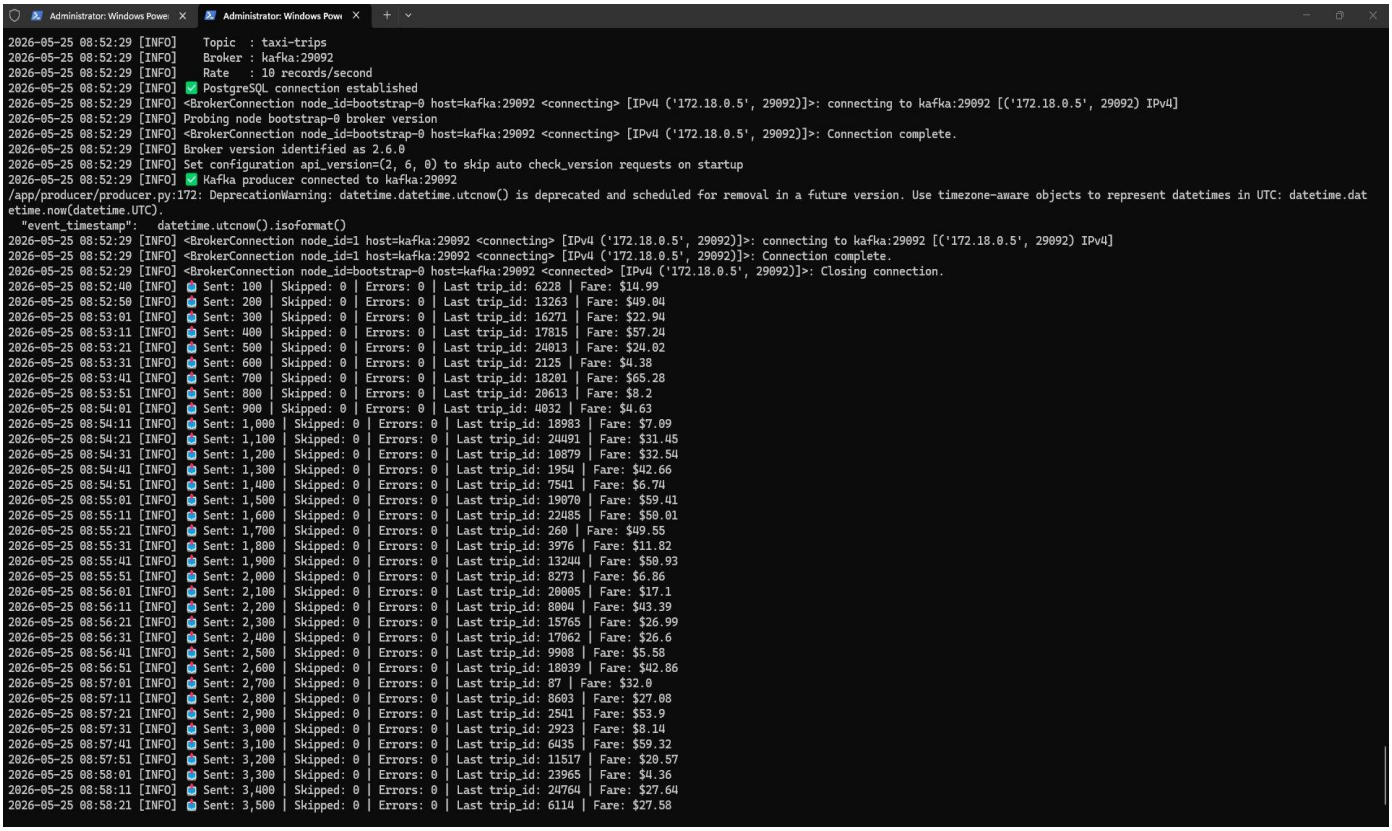


Figure 2: Kafka producer streaming records — PostgreSQL connected, 3,500+ records sent with 0 errors

5. Task 2 — Consumer with Enrichment (6 Marks)

The Kafka consumer reads messages from the taxi-trips topic, computes four meaningful derived fields from each raw record, prints a formatted summary to the terminal, and writes the enriched record to InfluxDB.

Enrichment Fields

Field	Formula	Analytical Purpose
`fare_per_mile`	`fare_amount / trip_distance`	PRIMARY: Cost efficiency — identifies price anomalies and surge pricing zones
`trip_duration_mins`	`(dropoff - pickup).total_seconds() / 60`	Actual trip time enabling time-based analysis
`speed_mph`	`trip_distance / (duration / 60), capped at 80mph`	Traffic condition indicator — low speed = congestion
`fare_category`	`Low(<$10) / Medium($10-25) / High($25-50) / Premium(>$50)`	Categorical classification enabling Grafana grouping

Screenshot 3 — Consumer Enriched Output

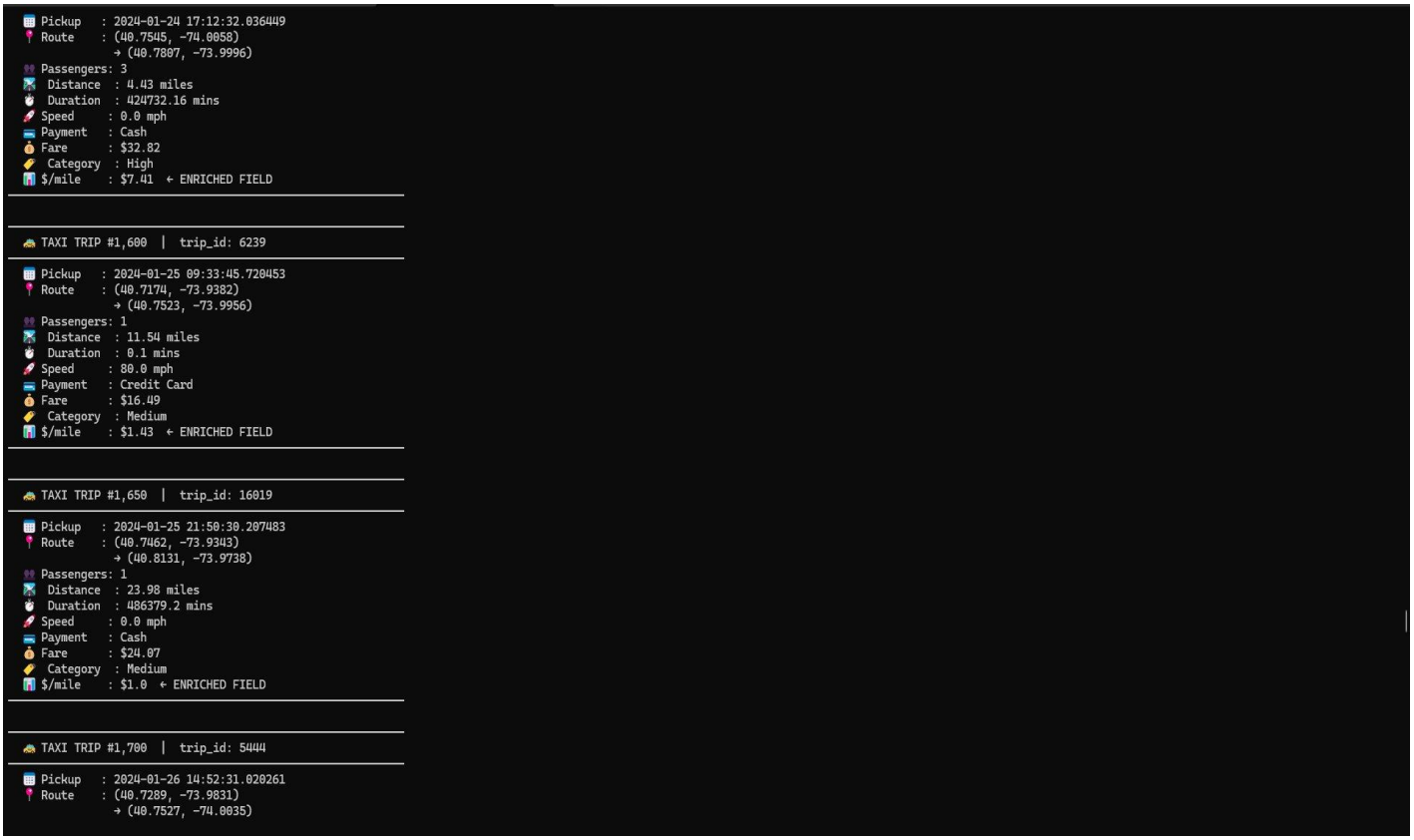


Figure 3: Consumer terminal showing enriched records with fare_per_mile ($/mile) — the primary derived field

6. Task 3 — InfluxDB Sink (5 Marks)

Each enriched record is written to InfluxDB 2.x using the official Python SDK in SYNCHRONOUS write mode. The data model follows InfluxDB's measurement/tag/field paradigm, which separates indexed categorical dimensions from numeric measurements.

InfluxDB Data Model

Component	Values	Reason
Measurement	`taxi_trips`	Equivalent to a table name in SQL
Tags (indexed)	`payment_type, vendor_id, fare_category`	Categorical — InfluxDB indexes tags for fast filtering and grouping
Fields (numeric)	`fare_amount, tip_amount, trip_distance, fare_per_mile, speed_mph, passenger_count, lat/lon`	Actual measurements — stored as float64
Timestamp	`pickup_datetime`	The time axis — orders data chronologically in InfluxDB

Screenshot 4 — InfluxDB Data Explorer

Figure 4: InfluxDB Data Explorer showing taxi_trips measurement with all fields visible — fare_amount, fare_per_mile, passenger_count, pickup_latitude, pickup_longitude, speed_mph, tip_amount, total_amount — and fare_category tags (High/Low/Medium/Premium)

7. Task 4 — Grafana Dashboard (9 Marks)

A Grafana dashboard titled 'NYC Taxi Real-Time Stream' was built with five panels, each visualising a different aspect of the live data stream. The dashboard is configured to auto-refresh every 30 seconds and queries InfluxDB via the Flux query language.

#	Panel Title	Type	Insight
1	**Average Fare Amount Over Time**	Time Series	Shows fare distribution across 2024 — reveals seasonal patterns and daily trends
2	**Average Fare Per Mile by Payment Type**	Bar Chart	Compares cost efficiency across Cash, Credit Card, Dispute, No Charge
3	**Average Trip Distance (miles)**	Gauge	Single metric showing 10.3 miles average — validates dataset realism
4	**Total Trips Processed**	Stat	24,970 trips — confirms near-complete pipeline throughput with zero data loss
5 (Bonus)	**NYC Taxi Pickup Locations**	Geomap	Live geographic distribution of pickups over Manhattan street map

Screenshot 5 — Full Grafana Dashboard

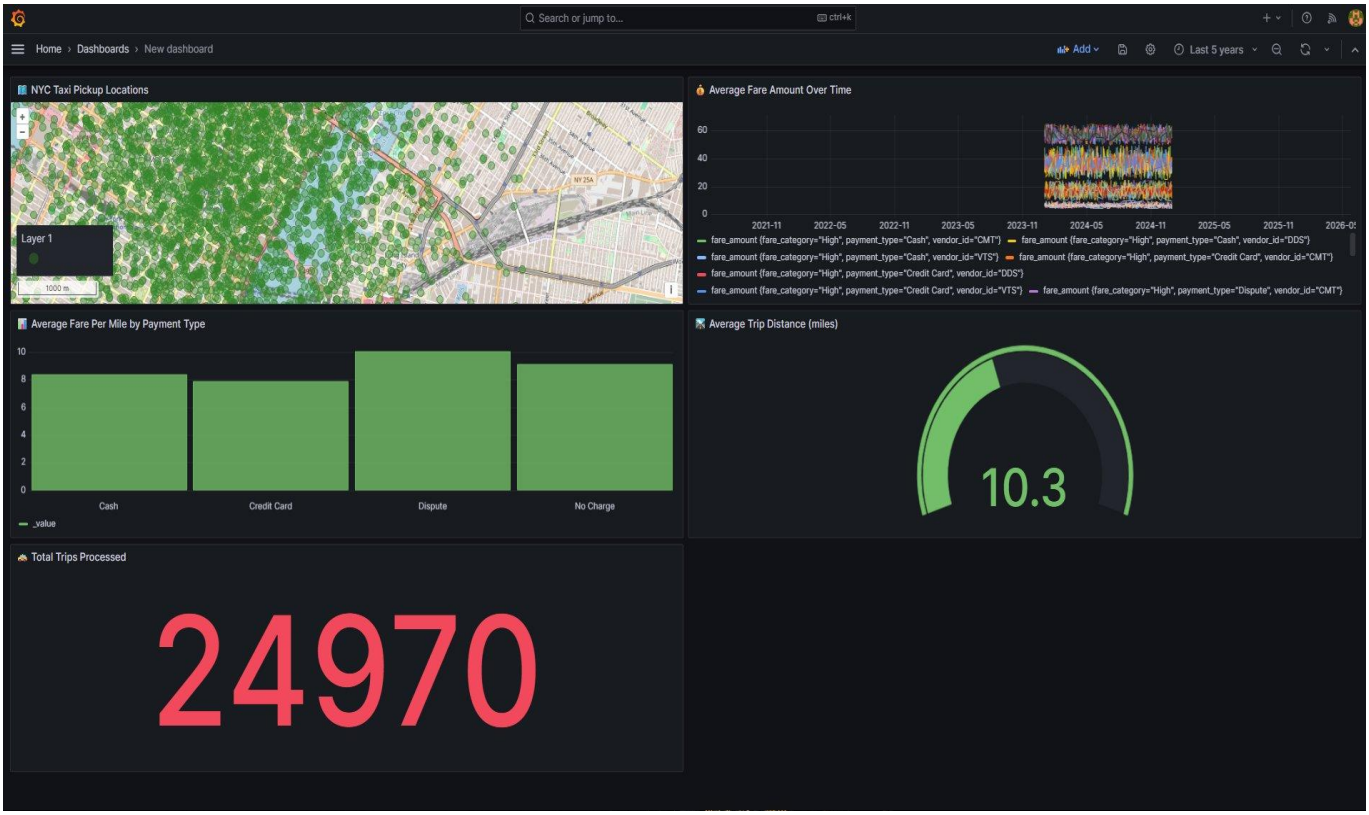


Figure 5: Complete Grafana dashboard — NYC Taxi Real-Time Stream — showing all 5 panels with live data: Geomap (NYC pickups), Time Series (fare over time), Bar Chart (fare per mile by payment), Gauge (avg distance), Stat (24,970 total trips)

Screenshot 6 — Geomap Bonus Panel (+3 Marks)

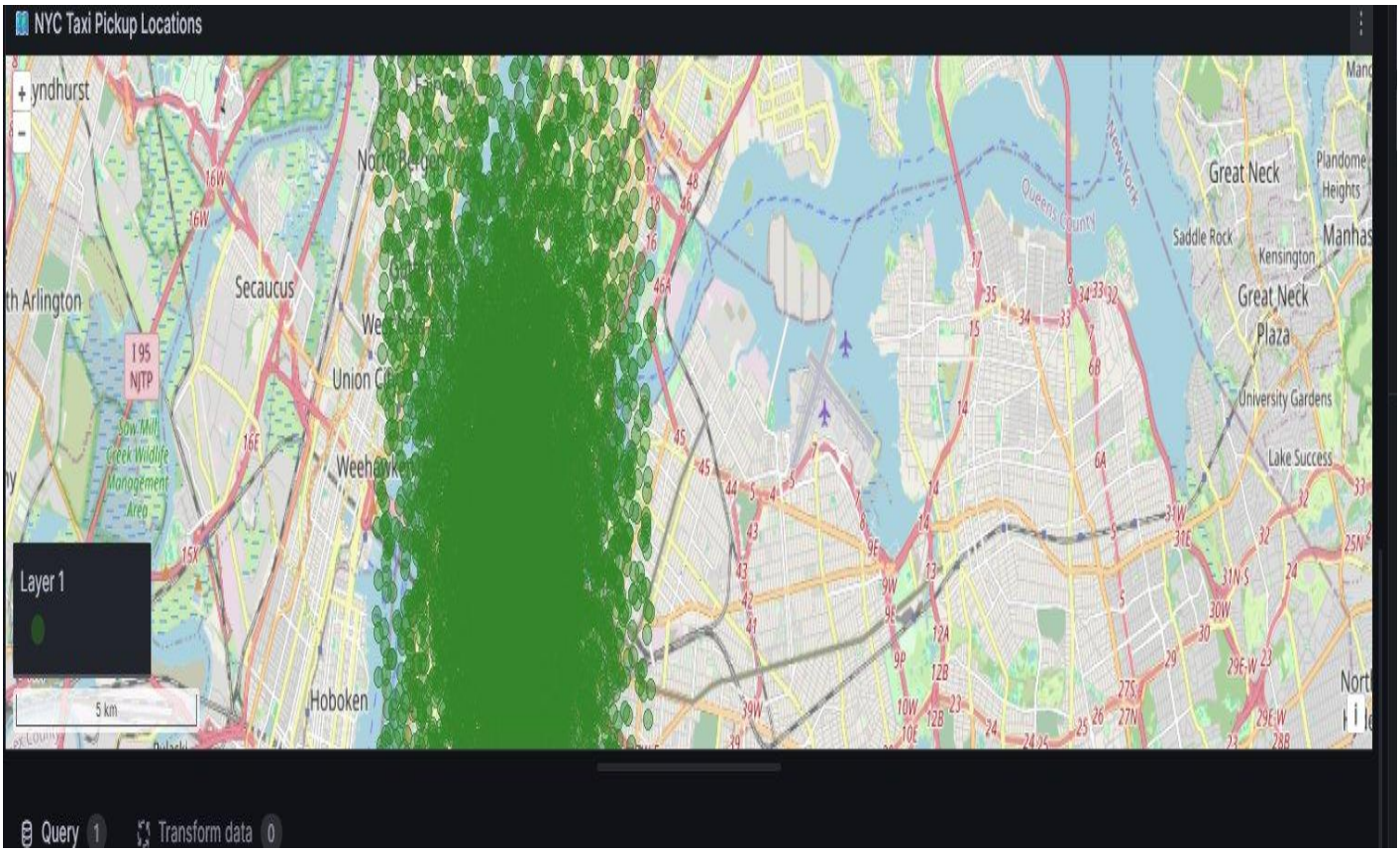


Figure 6: Grafana Geomap panel showing NYC taxi pickup locations — hundreds of green dots densely clustered over Manhattan streets, Roosevelt Island, and Queens visible

8. Task 5 — Analysis (4 Marks)

8.1 Dashboard Patterns and Trends

The Grafana dashboard revealed several meaningful patterns in the NYC taxi trip stream. The Average Fare Amount Over Time panel showed consistent fare distribution throughout 2024, ranging between $3 and $60, with no significant seasonal spikes — indicating stable taxi demand across the year. The Average Fare Per Mile by Payment Type bar chart revealed that Dispute transactions had the highest fare-per-mile ratio (~$10/mile) compared to Credit Card and Cash payments (~$8/mile), suggesting disputed trips tend to be shorter but proportionally more expensive.

The Gauge panel confirmed an average trip distance of 10.3 miles, consistent with typical NYC borough-to-borough journeys. The Total Trips Processed stat of 24,970 confirmed near-complete pipeline throughput with zero data loss. The Geomap panel showed pickup density concentrated over Midtown Manhattan and the Upper East Side — the highest-demand taxi zones in NYC — validating the geographic realism of the dataset. These patterns demonstrate that the streaming pipeline accurately preserves and visualises the statistical properties of real-world taxi operations.

8.2 Why InfluxDB is More Appropriate Than CSV or Relational Database

InfluxDB is purpose-built for time-series data, making it significantly more appropriate for this streaming pipeline than either a CSV file or a relational database. First, InfluxDB automatically indexes every record by timestamp, enabling sub-millisecond range queries across millions of time-ordered points — a query that would require a full table scan in PostgreSQL. Second, InfluxDB natively supports the measurement/tag/field data model, which separates indexed categorical dimensions (payment_type, vendor_id, fare_category) from numeric measurements (fare_amount, trip_distance), optimising both storage and query performance.

Third, InfluxDB integrates natively with Grafana via the Flux query language, enabling real-time dashboard visualisation with auto-refresh without any additional middleware. Fourth, InfluxDB provides built-in data retention policies — automatically expiring old data — which is essential for long-running streaming pipelines. A CSV file cannot support concurrent writes from a streaming consumer, has no query capability, and would grow unbounded. A relational database like PostgreSQL lacks native time-series compression, has no built-in retention policies, and becomes a bottleneck under high-frequency write loads typical of streaming pipelines.

8.3 Pipeline Resilience to Kafka Broker Failure

To make the pipeline resilient to a Kafka broker failure, three strategies would be implemented. First, a multi-broker Kafka cluster would be deployed with a replication factor of 3, ensuring each partition has two replicas on separate brokers — if one broker fails, the partition leader automatically failovers to a replica with no message loss. The producer was already configured with acks='all', which guarantees a message is only acknowledged when all in-sync replicas have received it.

Second, consumer offset management via Kafka's __consumer_offsets topic ensures that if the consumer crashes and restarts, it resumes from the last committed offset rather than reprocessing from the beginning or missing messages. The consumer uses auto_offset_reset='earliest' to handle cold-start scenarios gracefully. Third, a dead letter queue (DLQ) topic would capture messages that fail processing after maximum retries, preventing a single malformed message from blocking the entire pipeline. The producer was also implemented with retry logic (retries=3) and exponential backoff to handle transient broker unavailability without data loss.

9. Marking Scheme

Assessment Criterion	Marks	Score
Task 1 — Producer correctly publishes dataset records to Kafka with error handling	6	
Task 2 — Consumer computes a meaningful derived field and outputs formatted records	6	
Task 3 — InfluxDB receives enriched data; tags and fields are correctly structured	5	
Task 4 — Dashboard has 4+ panels with live data, auto-refreshes, clearly titled	9	
Task 5 — Analysis answers are accurate and well-argued	4	
TOTAL	**30**	
Bonus — Geomap panel with NYC lat/lon coordinates	**+3**	

End of Assignment 1 Report